Compilation Techniques: Laboratory

LLVM

Giuseppe Lomurno, University of Pisa

April 1st, 2026

What is LLVM?

A set of compiler and toolchain technologies

Leading compiler research technology

Adopted by industry at large scale

Modular and hackable

LLVM

Technologies

Provides code generation officially for C and C++ with Clang

Third-party drivers for many languages: C#, Haskell, Kotlin, Ruby, Rust, Swift, ...

All Apple technologies: operating systems, drivers, languages, ...

AMD and Intel C/C++ proprietary compilers are forks, Nvidia CUDA compiler

Supports many instruction sets: x86-64, ARM, RISC-V, Nvidia PTX, ...

LLVM

Toolchain

LLVM offers a C++ API to interact with the whole framework

Also exposes CLI commands for various steps: opt for optimizations, llc for static compilation, lli for JIT compilation ...

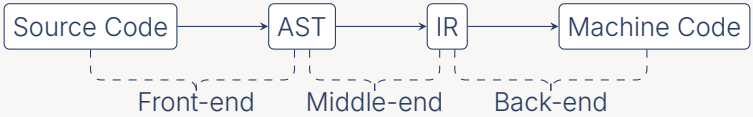
Related sub-projects:

- Clang: alternative frontend to GCC for C/C++
- LLDB: debugger
- LLD: linker
- MLIR: multi-level intermediate representation
- libc/libc++: implementation of C/C++ standard libraries

Compilation

Pipeline

Compiler are typically pipelines

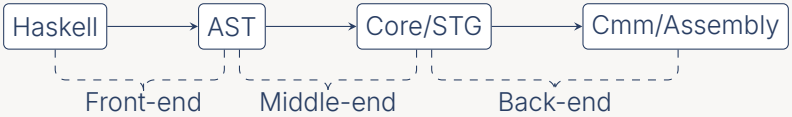


Front-end: Translation from source file to IR
Middle-end: Analyses and optimized IRs
Back-end: Generates target machine code from IR

Compilation

Pipeline

Compiler are typically pipelines



Front-end: Translation from source file to IR
Middle-end: Analyses and optimized IRs
Back-end: Generates target machine code from IR

LLVM

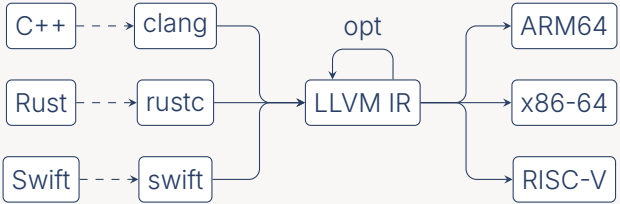
Pipeline

LLVM focuses on middle- and back-end

The front-end is language specific: translates your high-level language to LLVM IR

The optimizer analyses and transforms the LLVM IR

Back-end uses a machine-specific IR to generate machine code



LLVM

Intermediate Representation

A low-level programming language similar to assembly

Strongly typed

RISC-like, operates as-if registers are infinite

Three equivalent representations:

- Assembly: on-disk human-readable format
- Bytecode: on-disk binary format
- In-memory: in-memory binary format

LLVM

Intermediate Representation

```
define i32 @pow(i32 %x, i32 %y) {
start:
  br label %loop_start
loop_start:
  %i.0 = phi i32 [0, %start], [%i.new, %loop]
  %r.0 = phi i32 [1, %start], [%r.new, %loop]
  %done = icmp eq i32 %i.0, %y
  br i1 %done, label %exit, label %loop
loop:
  %r.new = mul i32 %r.0, %x
  %i.new = add i32 %i.0, 1
  br label %loop_start
exit:
  ret i32 %r.0
}
```

LLVM

Passes

The middle-end is itself a pipeline

Each stage is called a pass and can depend on other passes

The pass manager schedules passes according to dependencies

Passes transform LLVM IR to LLVM IR, or provide analysis

LLVM provides specialized passes skeletons depending on the scope: modules, functions, loops, call graph, ...

LLVM IR

Structure

While instructions are close to assembly, LLVM IR is still a high-level language

Hierarchical structure:

- Module: a single compilation unit, can be linked together; contains functions, global constants and symbol tables
- Function: defined by its arguments and a list of basic blocks
- Basic Block: a sequence of instructions

Typing

LLVM IR is strongly typed:

- Only explicit casting
- Everything is explicit annotated with its type

Strong typing enables some optimizations without extra analysis

Types are divided in subsets:

- Single value — types that are valid in registers: integers and vectors (for SIMD)
- First class — values that can be produced by instructions: single values, labels, aggregates, metadata
- Opaque — unspecified structure, used for external globals (like in C)
- Functions — single return with a list of parameters and/or varargs (printf)
- Void — no value, no size

Typing

Single value types

Integers

Integers of a specific bit width

`i1`, **`i32`**, **`i64`**, `i100`, `i1942625`, ...

Bytes

Raw data, cannot be determined if pointer, data, uninitialized, poison

`b1` • `b32` • `b64` • `b128` • ...

Floating-Point

Floating-point values of a specific bit width

`half` • `bfloat` • **`float`** • **`double`** • **`fp128`** • **`x86_fp80`** • **`ppc_fp128`**

Typing

Single value types

Pointers

Pointers to memory locations, can optionally reference a specific address space

Opaque but typed prior to LLVM 15 (or with an extension enabled), e.g. `i8*`)

`ptr` • `ptr addrspace(5)` • `ptr addrspace("A")`

Vectors

Vectors of primitive data that can be operated in parallel via SIMD

The number of elements is fixed (or a multiple depending on the architecture)

`<4 x i32>` • `<8 x float>` • `<4 x ptr>` • `<vscale x 4 x i32>`

Labels

Basic block label names

`label`

Typing

Aggregate types

Arrays

Sequential allocation of elements with the same type

Require a static size, not enforced while indexing

`[4 x i32]` • `[40 x i1]` • `[4 x [3 x float]]` • `[2 x [3 x [4 x int32]]]`

Structures

Collections of elements together in memory

Aligned according to the data layout by default, can be packed

Can be used as literals or defined as globals

`{i32, i32}` • `{i32, float, ptr}` • `<{i8, i32}>`

Typing

Other types

Void

Does not represent any value and has no size

`void`

Functions

Function signatures

Any type beside labels and metadata are valid both as arguments and return types

`i32 (i32)` • `{i1, i8, i23} (float)` • `i32 (ptr, ...)`

Constants

Simple

Booleans

`true` • `false` both of type `i1`

Integers

`4` • `-321` • `u0x8000` • `s0x8000`

Bytes

Equivalent to integers

Floating-point

`123.432` • `1.4325e+12` • `0x432ff973cafa8000`

Null pointer

`null`

Constants

Complex

Structures

```
{ i32 4, float 28.0, ptr null }
```

Arrays

```
[ i32 4, i32 11, i32 7 ] • c"Hello World0A00"
```

Vectors

```
< i32 4, i32 11, i32 7 > • splat (i32 11)
```

Zero initializer

```
zeroinitializer
```

Metadata

```
!{ !0, i32 0, ptr @global, ptr @function, !"str"}
```

Identifiers

Identifiers come in two types:

- Local — register names or types, prefixed by %
- Global — functions and global variables, prefixed by @

Three formats:

- Named values: `[%@][-a-zA-Z$._][-a-zA-Z$._0-9]*`
- Unnamed values: `%12`, `@2`
- Constants

Prefixes are used to avoid clashes with reserved words

Unnamed values are used to generate temporary variables avoiding symbol table conflicts

Modules

Definitions can have different linkage options

```
; Declare the string constant as a global constant.
@.str = private unnamed_addr constant [13 x i8] c"hello world\0A\00"

; External declaration of the puts function
declare i32 @puts(ptr captures(none)) nounwind

; Definition of main function
define i32 @main() {
  ; Call puts function to write out the string to stdout.
  call i32 @puts(ptr @.str)
  ret i32 0
}

; Named metadata
!foo = !{i32 42, null, !"string"}
```

Functions

Functions are lists of basic blocks between curly braces

A declaration must contain the name and the return type

Definitions can be annotated with optional information, e.g. linkage, calling convention

Parameters are passed as a list with mandatory typing and optional attributes

```
define noundef i64 @factorial(i64 noundef %n) unnamed_addr {
start:
  ...
bb3:
  ...
bb2:
  ...
}
```

SSA

LLVM, as for most optimizing imperative compilers, uses SSA for its IR

Static Single Assignment (SSA): every register is assigned exactly once

Statically: inside each function, for each register %x, there is only one "%x = ..."

Why? Simpler optimizations that preserve the SSA form

Optimizations are generally faster and multiple can be perfomed in single passes

Efficiently computed using a data-flow analysis: dominance frontiers

SSA

Example

```
uint64_t fib(uint16_t n) {
  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

```
uint64_t fib(uint16_t n) {









  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

SSA

Example

```
uint64_t fib(uint16_t n) {
  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

```
uint64_t fib(uint16_t n) {
  double x1 = pow(PHI, n);





  return (uint64_t) floor((x1 - pow(1 - PHI, n))/sqrt(5));
}
```

SSA

Example

```
uint64_t fib(uint16_t n) {
  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

```
uint64_t fib(uint16_t n) {
  double x1 = pow(PHI, n);
  double x2 = 1 - PHI;





  return (uint64_t) floor((x1 - pow(x2, n))/sqrt(5));
}
```

SSA

Example

```
uint64_t fib(uint16_t n) {
  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

```
uint64_t fib(uint16_t n) {
  double x1 = pow(PHI, n);
  double x2 = 1 - PHI;
  double x3 = pow(y, n);




  return (uint64_t) floor((x1 - x3)/sqrt(5));
}
```

SSA

Example

```
uint64_t fib(uint16_t n) {
  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

```
uint64_t fib(uint16_t n) {
  double x1 = pow(PHI, n);
  double x2 = 1 - PHI;
  double x3 = pow(y, n);
  double x4 = x1 - x3;



  return (uint64_t) floor(x4/sqrt(5));
}
```

SSA

Example

```
uint64_t fib(uint16_t n) {
  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

```
uint64_t fib(uint16_t n) {
  double x1 = pow(PHI, n);
  double x2 = 1 - PHI;
  double x3 = pow(y, n);
  double x4 = x1 - x3;
  double x5 = sqrt(5);


  return (uint64_t) floor(x4/x5);
}
```

SSA

Example

```
uint64_t fib(uint16_t n) {
  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

```
uint64_t fib(uint16_t n) {
  double x1 = pow(PHI, n);
  double x2 = 1 - PHI;
  double x3 = pow(y, n);
  double x4 = x1 - x3;
  double x5 = sqrt(5);
  double x6 = x4 / x5;

  return (uint64_t) floor(x6);
}
```

SSA

Example

```
uint64_t fib(uint16_t n) {
  return (uint64_t) floor((pow(PHI, n) - pow(1 - PHI, n))/sqrt(5));
}
```

```
uint64_t fib(uint16_t n) {
  double x1 = pow(PHI, n);
  double x2 = 1 - PHI;
  double x3 = pow(y, n);
  double x4 = x1 - x3;
  double x5 = sqrt(5);
  double x6 = x4 / x5;
  double x7 = floor(x6);
  return (uint64_t) x7;
}
```

SSA

Multiple assignments

```
float max(float a, float b) {
  float out;
  if (a > b)
    out = a;
  else
    out = b;
  return out;
}
```

The variable `out` must be statically assigned once!

SSA

Phi nodes

Introduce a phony node that abstracts over the conditional assignment

A ϕ-node is a list of pairs value-block label for each predecessor basic block

```
float max(float a, float b) {
  float out1, out2;
  if (a > b)
    out1 = a;
  else
    out2 = b;
  float out3 = phi(out1, out2);
  return out3;
}
```

ϕ-nodes must always be the beginning assignments in a basic block

Instructions

Terminators

Every basic block must end with a terminator

Return

Return a value to the caller

```
ret i32 5
ret void
```

Not Reachable

Marks an unreachable portion of code

```
unreachable
```

There are terminators for handling exceptions and resuming from exceptions

Terminators also can direct flow within a function

Branching

Jump to a specific block conditionally or unconditionally

```
br label %cont
br i1 %cond, label %IfTrue, label %IfFalse
```

Switch

Generalization of branching, similar to a switch in C

```
switch i32 %val, label %default, [ i32 0 %cond1
                                   i32 1 %cond2 ]
switch i32 0, label %dest [ ]
```

Operators

Integer Arithmetic Operators

Take two integers and return an integer of the same type, also work on integer vectors

Some allow additional flags between name and type to alter the semantics (e.g. overflow)

`%res = mul i32 %x, %y` • `%res = add nuw i8 %x, %y`

add Addition • **sub** Subtraction • **mul** Multiplication

udiv Unsigned quotient • **sdiv** Signed quotient • **urem** Unsigned remainder • **srem** Signed remainder

Floating-point Arithmetic Operators

Take two floats and return a float of the same type, also work on float vectors

Flags can enable unsafe transformations (e.g. assume no NaNs), and/or other rewritings

`%res = fneg float %x` • `%res = fadd float %x, %y` • `%res = fmul nnan float 2.0, %y`

fneg Negation • **fadd** Addition • **fsub** Subtraction • **fmul** Multiplication

fdiv Quotient • **frem** Remainder

Operators

Bitwise Binary Operators

Take two integers and return an integer of the same type, also work on integer vectors

Some allow additional flags between name and type to alter the semantics (e.g. overflow)

`%res = and i32 %x, %y` • `%res = shl nuw i8 %x, %y`

`shl` Left shift • **`lshr`** Shift right zero-fill • **`ashr`** Shift right with sign

`and` Conjunction • **`or`** Disjunction • **`xor`** Exclusive disjunction

Vector Operations

Extraction

Extracts from a specific index represented by an integer, always treated as unsigned

```
%res = extractelement <4 x i32> %vec, i32 0
%res = extractelement <vscale x 8 x i8> %vec, i64 12
```

Insertion

Inserts a value into a specific index represented by an integer, always treated as unsigned

```
%res = insertelement <4 x i8> %vec, i8 1, i32 0
```

Shuffling

Inserts a value into a specific index represented by an integer, always treated as unsigned

```
%res = insertelement <4 x i8> %vec, i8 1, i32 0
```

`shufflevector` builds a new vector from one or two vectors with a specific permutation mask

Aggregate Operations

Extraction

Extracts a value from an aggregate using constant indices

```
%res = extractvalue { i32, float } %agg, 0 ;yields i32
%res = extractvalue [ 4 x i32 ] %agg, 2 ;yields i32
%res = extractvalue { i32, { float, i8 } } %vec, 1, 0 ;yields float
```

Insertion

Inserts a value from an aggregate using constant indices

```
%res = insertvalue [ 4 x i32 ] %agg, i32 42, 0
%res = insertvalue { i32, { float, i8 } } %vec, float %val, 1, 0
```

Memory Operations

Stack Allocations

Allocates memory on the stack returning a pointer, automatically released on return

Size is a multiplier (1 if omitted) of the type size, optionally aligned to a constant

`%ptr = alloca i32` • `%ptr = alloca i8, i32 4`

`%ptr = alloca i32, i32 4, align 1024`

Stack Reads

Reads a value from the stack

There are options to specificy the atomicity and the alignment

`%val = load i32, ptr %ptr` • `%val = load i8, ptr %ptr, align 4`

Stack Writes

Stores a value in the stack

There are options to specify the atomicity and the alignment

`store i32 0, ptr %ptr` • `store i8 %val, ptr %ptr, align 4`

Pointer Arithmetic

Computes the address of a subelement of an aggregate, does not access memory

Computes the location of the i32 in the third pair

```
%ptr = getelementptr { i32, [ 12 x i8 ] }, ptr %agg, i64 2, i32 0
```

Computes the location of the 4th element of the vector in the third pair

```
%ptr = getelementptr { i32, <12 x i8> }, ptr %agg, i64 2, i32 1, i32 3
```

Computes the same by indexing partially

```
%ptr.1 = getelementptr { i32, <12 x i8> }, ptr %agg, i64 2
%ptr.2 = getelementptr { i32, <12 x i8> }, ptr %ptr.1, i64 0, i32 1
%ptr = getelementptr <12 x i8>, ptr %ptr.2, i32 0, i32 3
```

Casting

Casting are always explicit, convert a single value from one type to another

The syntax is always `op args <ty> <val> to <ty2>`

`trunc` Integer truncation • **`zext`** Integer zero extension • **`sext`** Integer sign extension
`fptrunc` Floating-point truncation • **`fpext`** Floating-point extension
`fptoui` Floating-point to unsigned integer • **`fptosi`** Floating-point to signed integer
`uitofp` Unsigned integer to floating-point • **`sitofp`** Signed integer to floating-point
`ptrtoint` Pointer to integer • **`inttoptr`** Integer to pointer • **`bitcast`** Cast without changing bits

Other Operations

Integer Comparisons

Compare integers (or integer vectors) with a given relation, returns a boolean `i1`

Relations are equality and signed/unsigned order: **`eq`**, **`ne`**, **`ugt`**, **`uge`**, **`ult`**, **`ule`**, **`sgt`**, **`sge`**, **`slt`**, **`sle`**

`%res = icmp eq i32 4, %x` • `%res = icmp ult i16 4, 2`

Floating-point Comparisons

Compare floats (or floats vectors) with a given relation, returns a boolean `i1`

Relations are equality and order, with QNaN allowed or not: **`uno`**, **`ueq`**, **`une`**, **`ugt`**, **`uge`**, **`ult`**, **`ule`**, **`ord`**, **`oeq`**, **`one`**, **`ogt`**, **`oge`**, **`olt`**, **`ole`**

`%res = fcmp oeq float 4.0, %x` • `%res = fcmp ult float 4.0, 2.0`

Other Instructions

Phi

Implementation of ϕ-nodes

```
%res = phi i32 [0, %block1], [%var, %block2]
%res = phi i32 [%var1, %block1], [%var2, %block2], [%var3, %block3]
```

Ternary Expressions

Ternary conditional without explicit branching, no short-circuiting

```
%res = select i1 true, i8 17, i8 %x
```

Function invokations

Simple function calls, also indirect via pointers

Optional flags may mark tail calling, calling convention, attributes (inlining, unwinding, ...)

```
%res = call i32 @test(i32 %argc)
%res = musttail call i32 @foo()
call void %foo(i8 signext 97)
call i32 (ptr, ...)@printf(ptr %msg, i32 12, i8 42)
```

Intrinsics

Intrinsic functions have predefined names and semantics and follow certain restrictions
Practically, extensions to LLVM without changing the language

Always start with the `llvm.` prefix

Used for overloading, by adding a suffix with the type, e.g. `.i8`

Varargs, code generation, standard library, arithmetic, vectors, matrices, atomic, ...

```
declare i64 @llvm.read_register.i64(metadata)
declare double @llvm.cos.f64(double %val)
declare void @llvm.memcpy.p0.p0.i32(ptr %dest, ptr %src, i32 %len, i1 %isvolatile)
declare {i16, i1} @llvm.sadd.with.overflow.i16(i16 %a, i16 %b)
```